

# Лабораторная работа №2

---

## Построение видео-пайплайна обработки субтитров с использованием n8n

---

### 1. Цель работы

---

Целью лабораторной работы является разработка автоматизированного пайплайна обработки видео, который:

- принимает видео по ссылке или файлом через Telegram;
- извлекает аудиодорожку;
- генерирует английские субтитры с использованием auto\_subtitle;
- выполняет перевод субтитров EN → RU с использованием LLM;
- встраивает русские субтитры в исходное видео;
- возвращает итоговый видеофайл пользователю в Telegram.
- Пайплайн должен быть реализован в виде оркестрируемого workflow и соответствовать требованиям задания.

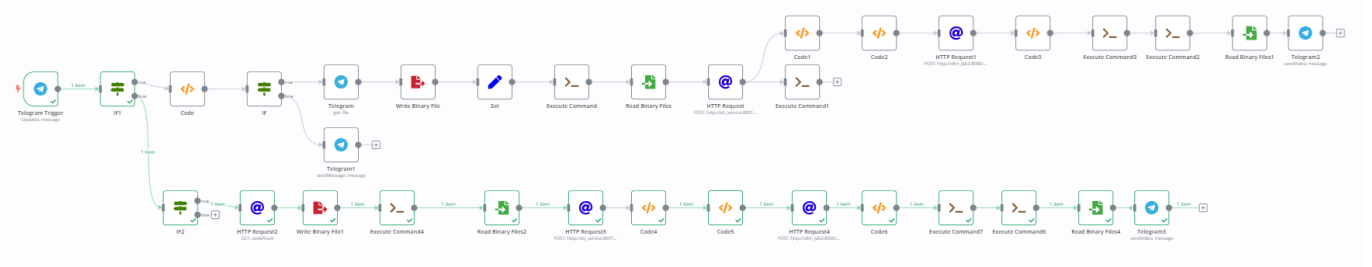
### 2. Архитектура решения

---

Общая архитектура решения построена на микросервисном подходе и включает следующие компоненты:

- n8n — оркестрация пайплайна и интеграция сервисов;
- Telegram Bot API — точка входа (файл или ссылка);
- ffmpeg — обработка мультимедиа (аудио и видео);
- auto\_subtitle — генерация субтитров на английском языке;
- vLLM (OpenAI-совместимый API) — перевод субтитров EN → RU;
- MinIO — объектное хранилище видеофайлов (режим работы по ссылке);
- Docker Compose — запуск и управление инфраструктурой.

Логическая схема пайплайна



Контейнеры

```

$ docker compose up -d
time="2025-12-14T20:36:09+04:00" level=warning msg="Found orphan containers [[minio_mc_lab2]] for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up."
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

```

The screenshot shows the Docker environment setup. The first command runs 'docker compose up -d', which outputs a warning about orphan containers and confirms the services are running. The second command, 'docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"', lists the running containers:
 

- Container stt\_service**: Running, ports 5678/tcp and 8000/tcp.
- Container minio\_lab2**: Running, ports 8000/tcp and 8001/tcp.
- Container vllm\_lab2**: Started, ports 8002/tcp and 8001/tcp.
- Container n8n\_lab2**: Started, ports 9000/tcp and 9001/tcp.

3. Используемые инструменты и версии

| Компонент     | Версия                         | Примечание                        |
|---------------|--------------------------------|-----------------------------------|
| n8n           | 0.222.0                        |                                   |
| Docker        | 4.54                           |                                   |
| ffmpeg        | 4.1                            |                                   |
| auto_subtitle |                                |                                   |
| vLLM          | 0.12.0                         |                                   |
| LLM модель    | Qwen/Qwen2.5-0.5B-Instruct-AWQ | Более новую версию не потянул GPU |
| MinIO         | latest                         |                                   |
| Python        | 3.10                           |                                   |

4. Описание пайплайна (Extract → Transform → Load)

В рамках данной лабораторной работы ETL-подход интерпретируется следующим образом:

- Extract — получение видео и аудио данных;
- Transform — генерация и перевод субтитров;
- Load — встраивание субтитров и возврат видео пользователю.

4.1. Extract

Поддерживаются два режима входа:

- Файл видео — полученный напрямую от пользователя в Telegram.
- Ссылка — HTTP-URL. (При тестировании использовалась MinIO)

```

rus1k@DESKTOP-2UM6UTD MINGW64 ~
$ /c/Users/rus1k/ngrok.exe http 5678

Session Status      online
Account             Ruslan Kashapov (Plan: Free)
Version             3.34.1
Region              Europe (eu)
Latency             63ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://0d68acb36db2.ngrok-free.app -> http://localhost:5678

Connections
  ttl    opn    rt1    rt5    p50    p90
    6     0     0.00  0.00  5.03  5.04

HTTP Requests
-----
20:37:34.520 +37 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK
20:34:52.713 +34 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK
20:28:32.537 +28 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK
20:17:28.221 +17 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK
20:16:59.652 +16 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK
20:16:16.404 +16 POST /webhook-test/34d3e98f-9f25-432e-a731-8589d0649c83/webhook 200 OK

```

Для

корректного взаимодействия телеграмма с n8n использовался ngrok.

Для обоих режимов видео сохраняется во временную директорию контейнера n8n:

/home/node/tmp/in.mp4

Извлечение аудио:

```
ffmpeg -y -i in.mp4 -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

## 4.2. Transform

### 4.2.1 Генерация EN субтитров (auto\_subtitle)

Для генерации английских субтитров используется отдельный сервис stt\_service, реализованный на Flask, внутри которого вызывается auto\_subtitle.

```

cmd = [
    "auto_subtitle",
    "--model", model,
    "--language", lang,
    "--output_dir", out_dir,
    "--output_srt", "True",
    "--srt_only", "True",
    audio_path,
]

p = subprocess.run(cmd, capture_output=True, text=True)

```

Результат работы:

subs\_en.srt

### 4.2.2 Перевод субтитров EN → RU (vLLM)

Перевод выполняется через OpenAI-совместимый API сервера vLLM:

POST /v1/chat/completions

Передаваемый prompt включает:

- инструкцию сохранить номера и таймкоды;
- текст исходных EN-субтитров.

Ограничения и решения:

- Из-за слабой GPU, используется слабая LLM, из-за чего может возвращать лишний текст и неправильный перевод.

### 4.2.3 Очистка и сборка SRT

Для приведения ответа модели к строгому формату SRT используется Code node в n8n:

удаляются вступительные и заключительные фразы;

восстанавливается формат:

index HH:MM:SS,mmm --> HH:MM:SS,mmm text

извлекаются только валидные SRT-блоки с помощью регулярных выражений.

Результат:

subs\_ru.srt

## 4.3. Load

Встраивание субтитров в видео

```
ffmpeg -y \  
-i in.mp4 \  
-vf "subtitles=subs_ru.srt:charenc=UTF-8" \  
-c:a copy \  
out.mp4
```

Итоговый файл out.mp4 содержит встроенные русские субтитры.

## 5. Обработка ошибок и устойчивость

---

Реализованы следующие меры:

- проверка наличия входного файла;

- контроль формата SRT перед встраиванием;
- защита от превышения размера HTTP-payload (413);
- работа только с файловыми путями вместо передачи бинарных данных в JSON;
- логирование ошибок ffmpeg, auto\_subtitle и vLLM.

Потенциальные точки сбоя:

- превышение контекста LLM;
- нестабильный формат ответа модели;
- сетевые ошибки Telegram/ngrok;
- недоступность MinIO.

## 6. Результаты работы

---

В результате выполнения лабораторной работы:

- реализован полностью автоматизированный видео-пайплайн;

- http://minio:9001/api/v1/download-shared-object/aHR0cDovLzEyNy4wLjAuMTo5MDAwL2xhYjIvbGFmI8xLm1wND9YLUFtei1BbGdvcml0aG09QVdTNC1ITUFDLVNIQTI1NiZYLUFtei1DcmVhZD50aWFsPTdXOFU0RkZjVIRDQk1RUETUNlk3JTJGMjAyNTEyMTQIMkZ1cy1YXN0LElMkZzMyUyRmF3czRfcmlvXzZCYZYLUFtei1EYXRIPTIwMjUxMjE0VDE2MTY0MVomWC1BbXotRXhwaXJlc300MzIwMCZYLUFtei1TZWN1cmI0eS1Ub2t1bj1leUpoYkdjaU9pSkVlV4TWJlc0luUjVjQ0k2SWtwWFZD5JkuZXlKaFkyTmxjM05MWlhraU9pSTNWemhWTkVaR1NWWlVRMEpOVVZCTFZEWlpOeUlzSW1WNGNDSTZNVGMyTIRjMk9URXhOaXdpY0dGeVpXNTBJam9pYlZsdWFXOGlmUS5mTXREdE8wY0c4TkN3eUJDZ2dDM2dsMk94Nm12bC05T3UxblBmWVQ0Vkt6RXhza2owZ1N5c3ZMTDNvN2V6YUVDR2dfSDhKbDFMWHhta2ZGbWN1ZGFNdYzYLUFtei1TaWduZWRIZWfKZXJzPWhvc3QmdmVyc2lvbklkPW51bGwmWC1BbXotU2lnbmF0dXJlPTE3YjNkNTIxNTU2Y2RiZjY2OWU4MDdmZDVhNGZiYTUwMjZkZWY1OWI0OGNkZGQyM2NhNmY0YTE3Mjk2ZDAzNTY

20:37 ✓

- субтитры генерируются с использованием auto\_subtitle;

```
[2025-12-14 16:36:54,735] INFO in app: auto_subtitle stderr tail= | 442M/461M [01:16<00:02, 7.56MiB/s]
96% | 443M/461M [01:17<00:02, 7.77MiB/s]
96% | 443M/461M [01:17<00:02, 8.06MiB/s]
96% | 444M/461M [01:17<00:02, 8.35MiB/s]
97% | 445M/461M [01:17<00:01, 8.54MiB/s]
97% | 446M/461M [01:17<00:01, 8.72MiB/s]
97% | 447M/461M [01:17<00:01, 8.98MiB/s]
97% | 448M/461M [01:17<00:01, 9.21MiB/s]
97% | 449M/461M [01:17<00:01, 9.49MiB/s]
98% | 450M/461M [01:17<00:01, 9.73MiB/s]
98% | 451M/461M [01:17<00:01, 9.94MiB/s]
98% | 452M/461M [01:18<00:00, 10.2MiB/s]
98% | 453M/461M [01:18<00:00, 10.5MiB/s]
98% | 454M/461M [01:18<00:00, 10.7MiB/s]
99% | 455M/461M [01:18<00:00, 10.7MiB/s]
99% | 456M/461M [01:18<00:00, 10.9MiB/s]
99% | 457M/461M [01:18<00:00, 11.0MiB/s]
99% | 458M/461M [01:18<00:00, 11.1MiB/s]
100% | 459M/461M [01:18<00:00, 11.1MiB/s]
100% | 461M/461M [01:18<00:00, 11.2MiB/s]
100% | 461M/461M [01:18<00:00, 6.13MiB/s]

0% | 0/11401 [00:00<?, ?frames/s]
24% | 2754/11401 [00:07<00:22, 388.88frames/s]
49% | 5634/11401 [00:14<00:14, 404.19frames/s]
75% | 8558/11401 [00:20<00:06, 415.06frames/s]
100% | 11401/11401 [00:23<00:00, 564.78frames/s]
100% | 11401/11401 [00:23<00:00, 494.23frames/s]

172.20.0.4 - - [14/Dec/2025 16:36:54] "POST /stt HTTP/1.1" 200 -
[2025-12-14 16:38:01,191] INFO in app: auto_subtitle rc=0
[2025-12-14 16:38:01,191] INFO in app: auto_subtitle stdout tail=Extracting audio from audio...
Generating subtitles for audio... This might take a while.

[2025-12-14 16:38:01,191] INFO in app: auto_subtitle stderr tail=
0% | 0/11401 [00:00<?, ?frames/s]
24% | 2754/11401 [00:05<00:16, 519.68frames/s]
49% | 5634/11401 [00:10<00:10, 528.17frames/s]
75% | 8558/11401 [00:17<00:05, 481.50frames/s]
100% | 11401/11401 [00:19<00:00, 638.62frames/s]
100% | 11401/11401 [00:19<00:00, 582.12frames/s]

172.20.0.4 - - [14/Dec/2025 16:38:01] "POST /stt HTTP/1.1" 200 -
```

- перевод выполняется локальной LLM;

```
APIServer pid=1 INFO: Application startup complete.
APIServer pid=1 INFO 12-14 08:40:08 [chat_utils.py:574] Detected the chat template content format to be 'string'. You can set '--chat-template-content-format' to override this.
APIServer pid=1 INFO 12-14 08:40:18 [loggers.py:236] Engine 000: Avg prompt throughput: 94.4 tokens/s, Avg generation throughput: 61.6 tokens/s, Running: 1 reqs, Waiting: 0 reqs, GPU KV cache usage: 1.3%, Prefix cache hit rate: 0.0%
APIServer pid=1 INFO: 172.20.0.4:56784 - "POST /v1/chat/completions HTTP/1.1" 200 OK
APIServer pid=1 INFO 12-14 08:40:28 [loggers.py:236] Engine 000: Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 37.6 tokens/s, Running: 0 reqs, Waiting: 0 reqs, GPU KV cache usage: 0.0%, Prefix cache hit rate: 0.0%
APIServer pid=1 INFO 12-14 08:40:38 [loggers.py:236] Engine 000: Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 0.0 tokens/s, Running: 0 reqs, Waiting: 0 reqs, GPU KV cache usage: 0.0%, Prefix cache hit rate: 0.0%
```

- итоговое видео успешно возвращается пользователю в Telegram.

## 7. Выводы

В ходе лабораторной работы был разработан практический пайплайн обработки видео с субтитрами, объединяющий современные инструменты оркестрации (n8n), мультимедийной обработки (ffmpeg) и LLM-модели.

Основные сложности были связаны с:

- ограничениями контекста LLM;
- нестабильностью формата ответа модели;
- передачей больших файлов между сервисами.

Все проблемы были успешно решены за счёт постобработки и использования объектного хранилища.

Возможные улучшения:

- автоматическое масштабирование vLLM;
- поддержка многоязычных субтитров;
- хранение истории обработок;
- добавление метрик времени выполнения шагов.